

AhamX Bodhi

Where I Becomes Infinite

Decoder-Only Architecture

Module 3.2: Transformer Architecture Deep-Dive
Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- Why the decoder-only design dominates modern LLMs
- Structure of a single decoder-only block
- How causal masking enables autoregressive generation
- Differences from the full encoder-decoder model
- GPT, Llama, Claude, and Mistral as case studies

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B2 – Understand
- **Difficulty:** L2 – Intermediate
- **Module:** 3.2 Transformer Deep-Dive
- **Prerequisite:** Encoder-Decoder Architecture



Learning Objectives

By the End of This Session You Will Be Able To

1. **Describe** the structure of a decoder-only Transformer block
2. **Explain** why causal masking is the defining constraint of this architecture
3. **Contrast** decoder-only with encoder-only and encoder-decoder models
4. **Identify** the in-context learning property unique to decoder-only models
5. **Map** GPT-4, Llama 3, Claude, and Mistral to this architecture



Prerequisites and Connections

You Should Already Know

- Scaled dot-product attention and causal masking
- Encoder-decoder architecture and its trade-offs
- Autoregressive generation and next-token prediction
- Residual connections and Layer Normalisation

This Concept Unlocks

- KV-Cache and inference optimisation
- In-context learning and few-shot prompting
- Pre-training on next-token prediction
- RLHF and instruction-tuning pipelines





Structure of a Decoder-Only Block

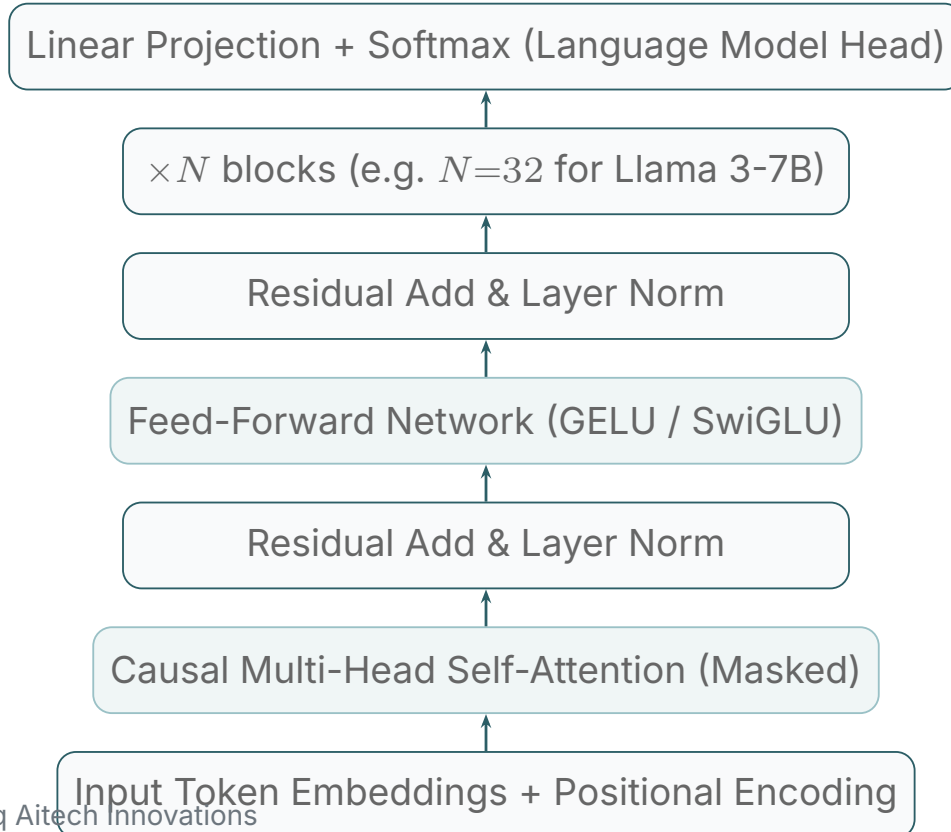
Two Sub-Layers Per Block (Compare: Decoder Has Three)

- **Causal Multi-Head Self-Attention:**
 - Every token attends to all *preceding* tokens (including itself)
 - Future tokens are masked out with $-\infty$ before the softmax
 - Q, K, V all come from the same sequence — no cross-attention layer
- **Feed-Forward Network:**
 - Two linear projections with a non-linearity (GELU, SwiGLU)
 - Inner dimension typically $4 \times d_{\text{model}}$
 - Applied independently at each token position
- Both sub-layers wrapped with residual connections and Layer Normalisation



Decoder-Only Block: Data Flow

Next-Token Probability Distribution





Causal Masking: The Defining Constraint

Why Masking is Central to This Architecture

- At training time, the model sees the full target sequence but a lower-triangular mask ensures token i can only attend to positions $1, \dots, i$
- This enforces the **autoregressive property**: the probability of the next token depends only on prior tokens — consistent with the language modelling objective
- Teacher forcing allows all positions to be trained in parallel (no sequential dependence at training time)
- At inference, new tokens are appended one at a time; the causal mask means prior positions' attention outputs need not be recomputed (enables KV-Cache)

Causal masking is what makes decoder-only models both trainable at scale and deployable efficiently at inference.



Decoder-Only vs. Encoder-Decoder

Property	Encoder-Decoder	Decoder-Only
Attention type	Bidirectional enc + Causal dec	Causal only
Cross-attention	Yes (encoder → decoder)	No
Tasks	Translation, summarization	Generation, chat, reasoning
Representative	T5, BART, Whisper	GPT-4, Claude, Llama 3
Training signal	Input-output pair	Next-token prediction
Context at generation	Full source available	Prior tokens only
In-context learning	Limited	Very strong



Modern Refinements in Decoder-Only Models

How Production LLMs Improve on the Original Design

- **Pre-Layer Normalisation (Pre-LN):** LayerNorm before (not after) attention; improves training stability and allows higher learning rates
- **Rotary Positional Embedding (RoPE):** Applied to Q and K inside attention; generalises better to longer contexts than learned absolute embeddings
- **Grouped-Query Attention (GQA):** Multiple query heads share K/V heads; reduces KV-cache size at inference — standard in Llama 2/3, Mistral
- **SwiGLU activation:** Replaces GELU in the FFN with a gated variant; consistently improves perplexity at the same parameter count
- **RMSNorm:** Replaces LayerNorm; simpler (no mean subtraction), faster, equally effective





GenAI Application: Conversational AI Systems

Decoder-Only Models Dominate Chat Applications

- **ChatGPT (OpenAI):** GPT-4 decoder-only model with RLHF alignment; over 100M weekly active users; conversation history as context window
- **Claude (Anthropic):** Constitutional AI-aligned decoder-only model with 200K-token context; excels at document analysis and multi-step reasoning
- **Gemini (Google):** Decoder-only model trained natively multimodal; integrated into Search, Workspace, Android
- **Llama 3 (Meta):** Open-weight decoder-only model enabling fine-tuning for domain-specific assistants without API dependency

The decoder-only architecture with next-token prediction pre-training is what enables these models to generalise to arbitrary conversational tasks.



GenAI Application: In-Context Learning and Few-Shot Prompting

Unique Capability of Decoder-Only Architectures

- **Zero-shot:** The model performs a task from a description alone — no examples needed. Decoder-only models trained on web-scale text learn task descriptions implicitly
- **Few-shot:** 3–10 input-output examples in the context window effectively “program” the model; no gradient updates required
- **Chain-of-thought prompting:** Prefixing with step-by-step reasoning examples dramatically improves performance on math and logic tasks
- **Tool use / function calling:** The model generates structured JSON to invoke external APIs — the entire interface is in the context window



GenAI Application: Agentic AI Systems

Decoder-Only Models as the Core of AI Agents

- **ReAct framework:** The model alternates between Reasoning (generating chain-of-thought) and Acting (generating tool calls) — all in one context window
- **AutoGPT / CrewAI:** Decoder-only LLM generates plans, tool calls, and self-evaluations in a loop; prior steps remain in context as memory
- **GitHub Copilot Agent:** Reads entire codebases within context window, plans multi-file edits, and generates diffs — all decoder-only generation
- **Claude Computer Use:** The model observes screen state as tokens, reasons over it, and generates action tokens to control the computer



GenAI Application: Code Generation

Decoder-Only Models for Software Development

- **GitHub Copilot:** GPT-4-based decoder-only model generating line, function, and file completions; 1.3M+ paying users; 46% of code accepted
- **Cursor AI:** Uses Claude and GPT-4 decoder models for “tab completion” that predicts multi-line edits based on surrounding code context
- **Devin (Cognition):** First “AI software engineer” using decoder-only LLM to generate complete solutions to SWE-bench software engineering tasks
- **Meta Code Llama:** Specialised fine-tune of Llama 3 on code; supports fill-in-the-middle (FIM) generation for completion of code at cursor position



Common Misconceptions

Clearing Up Confusion About Decoder-Only Models

- **“Decoder-only means no encoder”:** Technically correct, but the model still encodes context — it does so *within the same layers* via causal self-attention. The “encoder” intuition is just distributed across all layers.
- **“Decoder-only cannot understand context as well as encoder-only”:** For generation tasks, decoder-only models match or exceed encoder-only on understanding when scale is held constant. BERT-style models are better for tasks requiring explicit full bidirectional representations (classification, NER).
- **“All GPT models use the original decoder from the 2017 paper”:** Modern GPT models retain causal masking but use Pre-LN, RoPE, SwiGLU, and GQA — significant departures from the original decoder design.



Decoder-Only Architecture — Key Takeaways

1. Decoder-only models use only causal self-attention and FFN per block — no encoder and no cross-attention
2. Causal masking enforces the autoregressive property: token i sees only tokens $1, \dots, i$; future positions are masked to $-\infty$
3. The architecture excels at open-ended generation, in-context learning, and agentic tasks where no explicit source/target split exists
4. Modern refinements (Pre-LN, RoPE, GQA, SwiGLU, RMSNorm) now define production models like Llama 3, Mistral, and Claude
5. GPT-4, Claude, Gemini, and Llama — the dominant LLMs in production — all use decoder-only architectures trained on next-token prediction



What This Concept Unlocks

Next in Module 3.2

- Layer Normalisation
- Feed-Forward Networks in Transformers
- Full Transformer Implementation (coding)

Later in the Course

- KV-Cache and Inference Optimisation
- In-Context Learning and Prompting
- RLHF and Instruction Tuning
- LoRA Fine-tuning of Decoder-Only Models

Thank You

Decoder-Only Architecture | Module 3.2 | AhamX Bodhi